

domain.user 보안 조치 정리

대상 범위: com.team10.backend.domain.user, global.jwt, global.security, global.crypto **최초 작성일:** 2026-06-17 **갱신일:** 2026-06-18 — 코드 리뷰 중 발견된 항목 3건 추가 적용 ([신규] 표시), 주민등록번호 필드 AES-256-GCM 암호화 적용 반영

1. 개요

이 문서는 domain.user 영역(및 인증/보안 인프라인 global.jwt, global.security, global.crypto)에 적용된 보안 조치를 인증/인가, 토큰 관리, 로그인 방어, 1원 계좌인증, 개인정보 마스킹, 외부 연동, 입력값·파일 검증, 예외 처리, 설정 하드닝 순으로 정리한다.

이번 갱신에서는 코드 전수 점검 과정에서 발견된 보완점 3건을 실제로 수정해 반영했다 (2장, 6장, 8장, 10장에 [신규]로 표시). 모두 기존 동작을 깨지 않는 범위 내의 강화 조치다.

또한 같은 갱신 시점에 IdentityVerification.ocrResidentNumber 필드에 AES-256-GCM 컬럼 암호화를 추가로 적용했다 (6.1장). 이는 코드 리뷰 발견 사항이 아니라 별도로 계획·구현한 보안 강화 작업이다.

참고: CodefBankTransferService.sendOneWon()의 계좌번호·인증코드 로그는 마스킹되어 있지 않으나, 이는 실서비스 결함이 아니라 데모 시연을 위해 의도적으로 남겨둔 것이다 (관련 메모 확인됨).

2. 인증/인가 — JWT 기반 Stateless 인증

2.1 Access Token 구조 및 발급 (JwtProvider)

HMAC-SHA 서명 기반 Access Token을 발급한다. 클레임은 sub(userId), email, jti(토큰 고유 ID, 블랙리스트 식별용), iat, exp(발급 후 1시간)로 구성된다. jti를 클레임에 포함시켜 토큰 단위 무효화(블랙리스트)가 가능하도록 설계했다.

- parseTokenClaims()로 userId와 jti를 한 번의 파싱으로 함께 추출 — 서명 검증을 중복 수행하지 않도록 최적화
- parseUserIdIgnoreExpiry() — 만료된 Access Token이라도 서명이 유효하면 userId를 추출할 수 있어, Refresh Token 검증 시 활용

2.2 인증 필터 (JwtAuthenticationFilter)

- Authorization 헤더에서 Bearer 토큰을 추출해 서명을 검증하고 SecurityContext에 userId를 principal로 설정
- /api/v1/auth/logout 요청은 만료된 토큰도 허용 — 로그아웃 자체가 막히는 것을 방지
- 토큰의 jti가 블랙리스트(Redis)에 등록되어 있으면 SecurityContext를 비우고 필터 체인을 통과시켜, 최종적으로 인가 규칙(anyRequest().authenticated())에서 401로 차단되도록 함

2.3 SecurityConfig

- 세션을 STATELESS로 설정 — 서버에 세션 상태를 두지 않음
- CSRF, formLogin, httpBasic 비활성화 (REST API + JWT 조합에서 불필요한 공격면 제거)
- CORS는 cors.allowed-origins 속성으로 화이트리스트 관리, allowCredentials=true로 쿠키 기반 RT 전송 지원
- 인가 규칙: 회원가입/로그인/토큰갱신/Swagger/H2-console/에러 페이지만 permitAll, 그 외 전 요청은 인증 필요
- 커스텀 401/403 핸들러(JwtAuthenticationEntryPoint, JwtAccessDeniedHandler) 연결 — 일관된 JSON 에러 응답 제공
- JwtAuthenticationFilter를 UsernamePasswordAuthenticationFilter 앞에 삽입
- [신규] H2 Console 인가·프레임 정책의 프로필 가드 — 자세한 내용은 10장 참고

3. 토큰 무효화 및 세션 관리

3.1 Access Token 블랙리스트 (TokenBlocklistService)

Redis에 `blocklist:{jti}` 키로 토큰을 블랙리스트에 등록한다. TTL은 해당 Access Token의 잔여 유효시간으로 설정해, 불필요하게 오래 남지 않도록 한다. 회원탈퇴, 로그아웃, 비밀번호 변경 시 즉시 블랙리스트에 등록되어 탈취된 세션을 즉시 무효화한다 (`UserService.blacklistAccessToken()` 공통 처리).

3.2 Refresh Token Rotation (RefreshTokenService)

- Opaque(UUID) 토큰을 `refresh:{userId}` 키로 Redis에 저장, TTL 7일
- 재발급 시 Lua 스크립트(`GET_AND_DELETE_IF_MATCH`)로 조회+삭제를 원자적으로 처리 — 동시에 두 요청이 같은 RT로 재발급을 시도해도 하나만 성공 (race condition 방지)
- 불일치하는 RT가 제출되면 기존 RT를 삭제하지 않음 — 잘못된 요청이 정상 세션을 무효화하지 못하도록 함

3.3 Refresh Token 쿠키 보안 속성

RT는 응답 JSON 바디에 포함하지 않고(LoginRes, TokenRefreshRes에서 제외) HttpOnly 쿠키로만 전달한다. 쿠키는 다음 속성을 갖는다 (`AuthController.buildRtCookie()`):

- **HttpOnly** — JavaScript로 토큰 탈취(XSS) 방지
- **SameSite=Strict** — CSRF 방지
- **Secure** — 환경설정(cookieSecure)으로 제어, HTTPS 환경에서만 전송
- **Path=/api/v1/auth**로 범위 제한 — 불필요한 엔드포인트로 토큰이 전송되지 않도록 함

4. 로그인 보안

4.1 무차별 대입 공격 방지 (LoginAttemptService)

Redis에 `login:fail:{email}` 키로 실패 횟수를 관리한다. INCR+EXPIRE를 Lua 스크립트로 원자적으로 실행해 슬라이딩 윈도우 방식의 잠금을 구현했다.

항목	값
최대 실패 허용 횟수	5회
잠금 시간	30분
저장소	Redis (<code>login:fail:{email}</code>)

4.2 계정 존재 여부(User Enumeration) 노출 방지

`UserService.login()`은 비밀번호 일치 여부를 계정 상태(DORMANT/WITHDRAWN) 확인보다 먼저 검사한다. 이를 통해 “이메일은 존재하지만 비활성 상태”라는 정보가 공격자에게 노출되지 않도록 한다.

4.3 비밀번호 저장

BCryptPasswordEncoder로 단방향 해싱 후 저장한다.

4.4 계정 상태 변경 시 세션 강제 종료

- `withdraw()` (회원탈퇴) — RT 삭제 + AT 즉시 블랙리스트 등록
- `changePassword()` (비밀번호 변경) — RT 삭제 + AT 즉시 블랙리스트 등록 (탈취된 세션 강제 종료)
- `logout()` — RT 삭제 + AT 블랙리스트 등록 (AT 파싱 실패에도 관용적으로 처리)

5. 1원 계좌인증 보안 (OneWonVerificationService)

실제 금융 거래(1원 송금)가 발생하는 기능이므로 중복 송금 방지와 무차별 대입 방지에 특히 신경 썼다.

방어 항목	정책
인증 코드	SecureRandom 기반 4자리 숫자, TTL 10분
검증 시도 제한	최대 5회 실패 시 코드 즉시 폐기(LOCKED)
일일 요청 한도	사용자당 하루 10회 (자정 기준 TTL 1일)
중복 요청 방지	SET NX 기반 10초 단기 락 — 동시 요청으로 인한 중복 실제 송금 방지

- 일일 카운터는 Lua 스크립트로 “최초 INCR 시에만 EXPIRE” 하도록 구현 — 매 요청마다 TTL이 갱신되어 한도가 무력화되는 것을 방지
- 송금 실패 시 보상 처리: `deleteCode()`(인증 코드 삭제), `decrementDailyCount()`(일일 카운터 롤백)로 사용자에게 불리한 카운트 차감이 남지 않도록 함

6. 개인정보 마스킹 / 데이터 보호

주민등록번호 등 민감정보는 DB 저장 단계(필드 암호화 + 마스킹), 애플리케이션 로그 출력 단계, 그리고 [신규] ORM 디버거 로그 단계까지 보호 조치를 적용한다.

6.1 OCR 주민등록번호 필드 암호화 (CryptoStringConverter)

`IdentityVerification.ocrResidentNumber` 필드에 JPA `AttributeConverter`를 적용해, DB에는 항상 암호문만 저장되고 애플리케이션 코드(엔티티 getter 등)에서는 평문으로 투명하게 다뤄지도록 했다.

- **알고리즘:** AES-256-GCM. 저장 형식은 `Base64( IV(12byte) || ciphertext || authTag(16byte) )` — 매 암호화마다 `SecureRandom`으로 새 IV를 생성하므로 동일한 평문이라도 매번 다른 ciphertext가 생성된다. 따라서 이 컬럼은 동등(=) 검색이 불가능하며, 적용 대상은 평문 검색이 필요 없는 민감정보(주민등록번호)로 한정했다.
- **적용 방식:** `CryptoStringConverter(global.crypto)`를 `@Component + @Converter(autoApply = false)`로 등록하고, `ocrResidentNumber` 필드에 `@Convert(converter = CryptoStringConverter.class)`를 명시적으로 지정. Hibernate가 INSERT/UPDATE/SELECT 시점에 컨버터를 투명하게 호출한다.
- **키 관리:** 생성자에서 `app.encryption.key`(Base64 인코딩된 32바이트 AES-256 키)를 주입받으며, 디코딩 결과가 32바이트가 아니면 기동 시점에 즉시 예외를 던진다(fail-fast).
 - dev: `application-secret.yml`에 실제 키 보관 (gitignore 처리, 커밋되지 않음)
 - test: `application-test.yml`에 테스트 전용 키를 리터럴로 직접 기재 (휘발성 H2 테스트 데이터만 다루므로 안전)
 - prod: `ENCRYPTION_KEY` 환경변수로 주입, 기본값 없음 — 미설정 시 기동 실패. 운영 배포 인프라(Dockerfile, CI/CD 배포 워크플로) 자체가 레포에 아직 없어 `application-prod.yml`의 안내용 플레이스홀더는 미사용 코드로 판단해 제거했으며, 이는 fail-fast 동작 자체에는 영향을 주지 않는다
- **마스킹과의 관계:** 6.2의 마스킹은 본인확인이 완료된 이후(성공/실패 무관) 값을 부분 마스킹 문자열로 덮어써 평문 흔적을 지우는 조치다. 이 암호화는 본인확인이 진행 중이라 아직 마스킹되지 않은 구간에도 DB 컬럼 자체가 평문으로 남지 않도록 보강한 것 — 두 조치가 같은 필드를 서로 다른 시점에서 함께 보호한다.

6.2 검증 완료 후 마스킹

- `IdentityVerification.maskResidentNumber()` — 주민번호를 앞 6자리 + “-*****” 형태로 마스킹. `completeGovernmentVerification()`(인증 성공) 및 `fail()`(인증 실패) 양쪽 경로 모두에서 호출되어, 성공/실패 관계없이 평문 주민번호가 DB에 남지 않도록 함. 마스킹 메서드는 idempotent하게 구현됨

6.3 애플리케이션 로그 출력 단계

- `MockGovernmentVerifyService.mask()` — 정부 본인확인 시뮬레이션 로직의 로그에서 동일한 마스킹 형식 적용

- `CodefOcrClient.maskIdentity()` — OCR 결과 필드 누락 경고 로그 등에서 주민번호 마스킹

6.4 [신규] ORM 레벨 바인드 파라미터 TRACE 로깅 비활성화

코드 점검 중 `application-dev.yml`에서 Hibernate의 `org.hibernate.orm.jdbc.bind / jdbc.extract` 로그 레벨이 TRACE로 켜져 있던 것을 발견해 비활성화했다.

- **문제:** Hibernate는 TRACE 레벨에서 SQL에 바인딩되는 모든 파라미터 값을 콘솔에 평문으로 출력한다. `IdentityVerification` 같은 엔티티는 주민등록번호를 다루므로, 6.2에서 적용한 DB 마스킹·6.3의 로그 마스킹을 모두 우회해 개발 환경 콘솔/로그 파일에 평문 PII가 그대로 남는 경로가 있었다.
- **조치:** 해당 로그 레벨을 제거하고, 대신 `show-sql + format_sql`로 SQL 문 자체(바인드 값 제외)만 확인 가능하도록 유지했다.
- **적용 위치:** `application-dev.yml`
- **참고:** 현재는 6.1의 컬럼 암호화가 먼저 적용된 뒤 `ciphertext`가 바인딩되므로, 설정 TRACE가 다시 켜져도 `ocrResidentNumber` 자체는 평문으로 노출되지 않는다. 다만 다른 평문 컬럼까지 보호하는 것은 아니므로 이 조치(TRACE 비활성화)는 별도로 유지한다.

6.5 의도적 예외 — 1원 송금 로그

`CodefBankTransferService.sendOneWon()`의 성공/실패 로그는 계좌번호와 인증코드를 마스킹하지 않고 그대로 출력한다. 이는 보안 결함이 아니라 데모 시연 편의를 위해 의도적으로 남긴 것으로, 별도 메모로 확인되어 있다. 향후 정식 운영 전환 시에는 마스킹 적용을 검토할 필요가 있다.

7. 외부 연동 보안 (CODEF / PortOne)

본인확인(PortOne) 및 1원 계좌인증(CODEF) 등 외부 API 연동에서는 트랜잭션 경계를 명확히 분리해 장애 상황에서도 데이터 일관성이 깨지지 않도록 했다.

- 외부 API를 호출하는 구간은 `@Transactional(propagation = Propagation.NOT_SUPPORTED)`로 처리 — 외부 호출 지연이 DB 커넥션을 점유한 채 트랜잭션을 묶어두지 않도록 분리
- 실패 기록은 `@Transactional(propagation = Propagation.REQUIRES_NEW)`로 별도 트랜잭션에서 처리 (`VerificationSessionRecorder.markFailedInNewTransaction()`) — 외부 호출 실패로 상위 트랜잭션이 롤백되어도 실패 사실 자체는 별도로 커밋되어 기록이 남음
- `GovernmentVerifyTimeoutException`은 `RuntimeException`으로 설계해, `@Transactional` 경계에서 자동 롤백 대상이 되도록 함
- CODEF/PortOne 인증 토큰은 별도 클라이언트(`CodefAuthClient` 등)에서 관리하며, 호출 시점에만 `Authorization` 헤더로 부착

8. 입력값 검증 및 파일 업로드 검증

8.1 Bean Validation

`@Valid` 및 `@NotBlank` 등 Bean Validation 애노테이션을 다음 7개 요청 DTO에 적용했다: `TokenRefreshReq`, `UserCreateReq`, `OneWonStartReq`, `ChangePasswordReq`, `ConsentUpdateReq`, `LoginReq`, `OneWonVerifyReq`.

8.2 파일 업로드 검증 (`IdentityVerificationService.validateImage()`)

- 최대 업로드 크기 10MB 제한 (`OCR_IMAGE_TOO_LARGE`)
- Content-Type 화이트리스트 — `image/jpeg`, `image/png`만 허용 (`OCR_IMAGE_INVALID_TYPE`)
- 빈 파일 업로드 차단 (`OCR_IMAGE_REQUIRED`)
- [신규] 매직바이트(파일 시그니처) 검증 — `hasValidImageSignature()`

Content-Type 헤더는 클라이언트가 요청 시 임의로 지정할 수 있는 값이라, 위 화이트리스트 검사만으로는 실제로는 다른 형식의 파일을 image/jpeg로 위장해 올리는 것을 막지 못한다. 이를 보완하기 위해 파일의 실제 앞부분 바이트(매직바이트)가 JPEG(FF D8 FF) 또는 PNG(89 50 4E 47 0D 0A 1A 0A) 시그니처와 일치하는지 한 번 더 검증하도록 추가했다. Content-Type과 매직바이트 중 하나라도 불일치하면 동일하게 OCR_IMAGE_INVALID_TYPE으로 거부한다. **적용 위치:** IdentityVerificationService.java — validateImage() → hasValidImageSignature() → matchesSignature()

참고(경미한 보완점, 미적용): Spring 레벨의 spring.servlet.multipart.max-file-size 설정은 application-dev.yml에만 존재하고 application-prod.yml에는 없다. 애플리케이션 레벨 검증(validateImage())이 실질적인 위험은 커버하지만, defense-in-depth 관점에서 prod 프로필에도 서버 레벨 제한을 추가하면 더 견고해진다.

8.3 인가 패턴

모든 컨트롤러에서 @AuthenticationPrincipal Long userId를 사용해 인증 토큰에서 추출한 userId만 신뢰한다. 클라이언트가 요청 본문에 직접 넣은 userId를 신뢰하는 코드는 없다.

9. 예외 처리 보안 (GlobalExceptionHandler)

- BusinessException, MethodArgumentNotValidException, ConstraintViolationException, MissingServletRequestParameterException, MethodArgumentTypeMismatchException을 각각 적절한 예러 코드로 변환해 응답
- 그 외 예측하지 못한 모든 Exception은 catch-all 핸들러에서 서버 로그에는 상세 내용을 남기되, 클라이언트에는 일반적인 INTERNAL_SERVER_ERROR만 반환 — 스택 트레이스나 내부 구조가 클라이언트에 노출되는 것을 방지
- Security Filter Chain에서 발생하는 401/403은 이 핸들러가 다루지 않고, 상류의 JwtAuthenticationEntryPoint / JwtAccessDeniedHandler에서 별도로 처리

10. 설정 및 CORS 하드닝

- CORS 허용 출처는 cors.allowed-origins 설정값 기반 화이트리스트로 관리 (전체 허용 와일드카드 미사용)
- 허용 메서드는 GET/POST/PUT/PATCH/DELETE/OPTIONS로 제한
- JWT 서명 키, DB/Redis 자격증명 등 민감 설정값은 환경설정(application*.yml) 외부 변수로 분리 관리
- [신규] H2 Console 인가·iframe 허용의 프로필 가드 (SecurityConfig)

기존에는 H2 콘솔의 permitAll() 인가와 frameOptions().sameOrigin() 헤더 설정이 프로필과 무관하게 적용되어, dev 편의를 위한 설정이 prod에도 그대로 노출될 위험이 있었다. environment.acceptsProfiles(Profiles.of("test"))로 분기해, dev/test 프로필에서만 /h2-console/**를 permitAll + 동일 출처 프레임 허용으로 열어주고, 그 외 프로필(prod 등)에서는 인증이 필요한 기본 상태를 유지하며 frameOptions().deny()로 클릭재킹 방어도 그대로 유지하도록 수정했다. **적용 위치:** SecurityConfig.java — filterChain()

11. 종합 정리

domain.user 영역은 인증(JWT Stateless), 토큰 무효화(블랙리스트+RT Rotation+쿠키 보안 속성), 로그인 방어(잠금+계정열거 방지), 금융 기능 보호(1원 인증 한도·락), 개인정보 보호(주민번호 컬럼 암호화 + 마스킹 + ORM 로그 차단), 외부 연동 안정성(트랜잭션 분리), 입력 검증(Beans Validation + 파일 업로드 매직바이트 검증), 설정 하드닝(프로필별 H2 Console 가드), 예외 처리(정보 노출 방지)까지 다층적으로 보안 조치가 적용되어 있다.

현재 시점에서 확인된 유일한 의도적 예외는 1원 송금 로그의 비마스킹 처리(데모 목적)이며, 추가로 보완하면 좋을 경미한 항목은 prod 프로필의 Spring 레벨 multipart 업로드 크기 제한 설정 정도다.

12. 변경 이력 (2026-06-18 코드 점검분)

항목	변경 전	변경 후	관련 장
OCR 주민등록번호 저장 방식	DB에 평문 저장 (검증 완료 후에만 마스킹)	AttributeConverter 기반 AES-256-GCM 컬럼 암호화 추가 — 검증 진행 중에도 DB에는 암호문만 존재	6.1
Hibernate 바인드 파라미터 로깅	dev 환경에서 TRACE — PII 평문 노출 가능	TRACE 로깅 제거, SQL 문만 확인 가능하게 유지	6.4
OCR 이미지 업로드 검증	Content-Type 헤더만 검사 (위장 가능)	매직바이트(파일 시그니처) 검증 추가	8.2
H2 Console 노출 설정	프로필 무관하게 permitAll + sameOrigin 적용	dev/test 프로필에서만 적용, 그 외엔 기본 보안값 유지	10